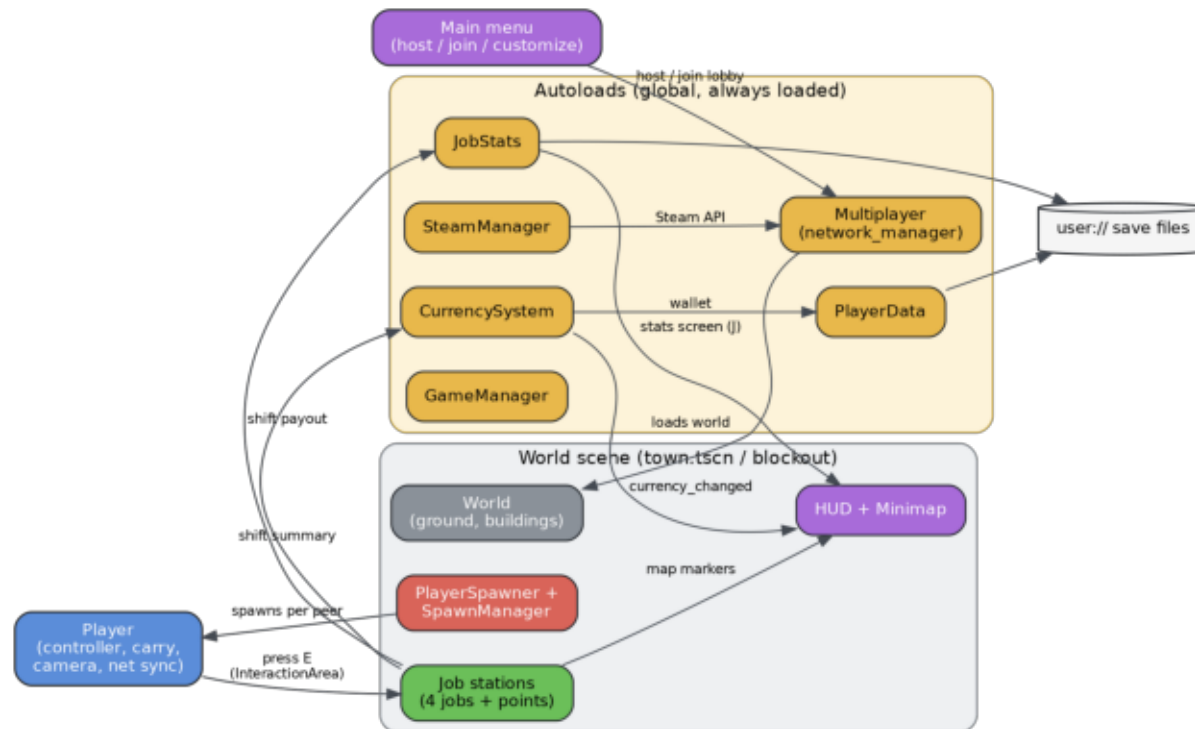


City Critters — Game Systems

How every system connects. Cozy multiplayer life-sim: work jobs, earn money, live in a critter city.



READ THIS MAP FIRST. Yellow boxes are autoloads — global scripts alive from launch to quit in every scene. The world scene (town or the St Guy blackout) contains the job stations, HUD, and a spawner that creates one player per connected peer. Everything the player earns flows into the autoloads and gets saved to disk.

COLOR KEY. Amber = autoloads / global state. Blue = player. Green = jobs. Purple = UI. Red = multiplayer. Gray = world scenes. Later pages zoom into each color.

Pages: 2 overview - 3 player - 4 shift lifecycle - 5 job catalog - 6 economy and stats - 7 minimap - 8 multiplayer - 9 prototypes.

PLAYER

You control a critter in third person. A state machine (idle, walk, run, running jump) drives root-motion animations — the animation itself moves the body, so feet never slide. Movement with a forward component steers the critter toward where you push; pure sideways or backward input strafes while facing the camera.

ECONOMY & STATS

One wallet (PlayerData.currency) fed through CurrencySystem so the HUD always updates. JobStats records every shift: how many, how much, best shift, and a performance rating per job. Both persist to disk and survive restarts.

MULTIPLAYER

Steam lobbies (GodotSteam). The host's machine spawns one player per peer; only your own machine simulates your critter, everyone else sees a replicated copy. With no Steam running the game still boots solo — the spawner just drops in one local player.

JOBS

Four minigame jobs share one skeleton: clock in at a station, tasks arrive on a timer, do them fast for tips, get paid once when the shift ends. Food Service and Retail are carry-things-around jobs, Delivery sends you running across the whole city, Corporate is a desk overlay where you audit expense reports.

UI

A HUD shows money live and hides a job-record screen behind the J key. The round minimap spins with the camera — whatever you're facing is up — showing every job site as a colored dot and the current objective as a pulsing diamond that never leaves the rim.

WORLDS & PROTOTYPES

Two playable maps: the modular town (parametric building and street assemblers) and the St Guy blackout (a whole village generated from a map file, no art assets). NPCs, hobbies (fishing), housing/furniture, and dialogue exist as early skeletons waiting for content.

Player: movement, animation, carrying

One CharacterBody3D. The owner peer simulates; everyone else replays replicated state.



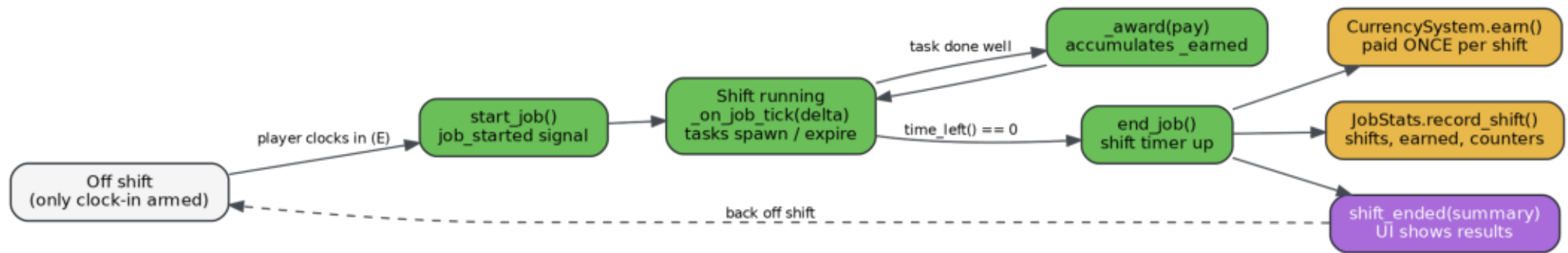
HOW IT MOVES. Movement is root motion: the state machine (LimboHSM) picks a clip (idle / walk / run / run_jump), the AnimationTree plays it, and player_controller turns the animation's built-in displacement into physics velocity every tick. Forward-fan input (W, W+A, W+D, any stick angle with a forward component) rotates the model toward travel direction and plays the forward clip; pure side/back input strafes camera-locked. Shift toggles run; running jump only from a run.

CARRYING. CarryComponent holds a list of item names plus little meshes stacked above the critter's hand. Jobs call player.get_carry() then pick_up / drop_item. Capacity 1 by default (new pickup swaps).

BUSY MODE. set_busy(true) freezes physics and camera and frees the mouse — used while a job overlay or menu is open.

The shift: one lifecycle for every job

job_base.gd owns the loop below; each job only fills in what a 'task' means.

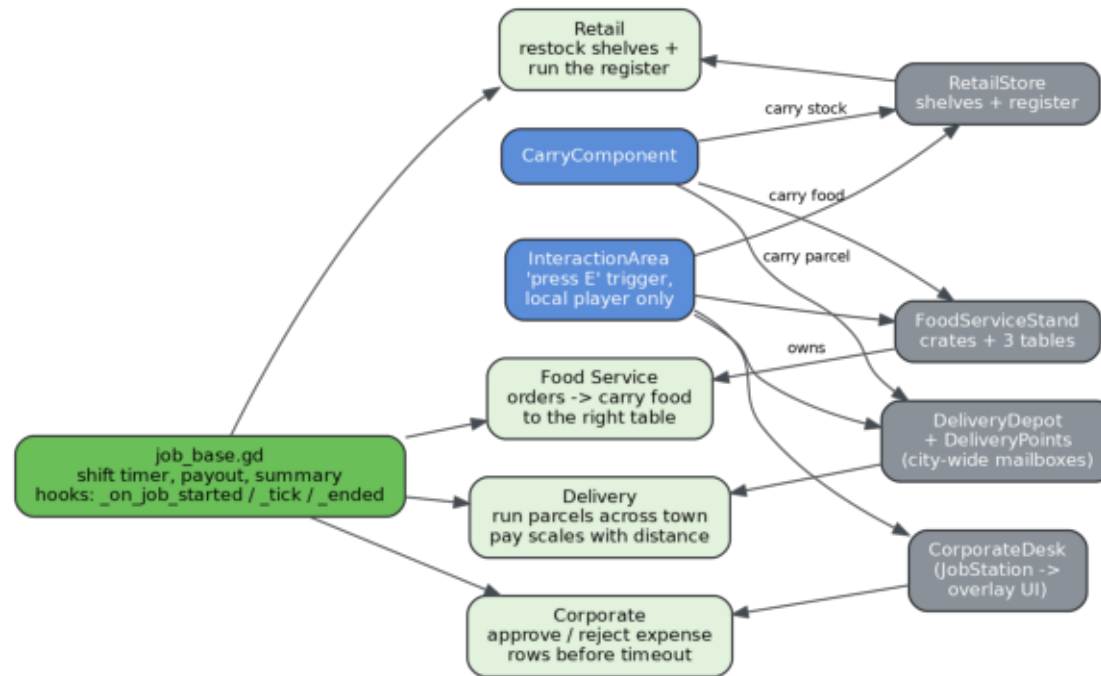


WHY ONE BASE CLASS. Timer, payout accumulation, the pay-once guarantee, signals, and stats recording live in job_base.gd. A new job overrides four hooks (`_on_job_started`, `_on_job_tick`, `_on_job_ended`, `_build_summary`) and calls `_award/pay)` whenever the player does something well — nothing else to wire. Stats and currency come free.

GATING. Stations arm their interactables only during a shift: before clock-in only the register/clock-in area responds to E; when the shift ends everything disarms and any carried items are dropped. The InteractionArea prompt ('press E') renders only for the local player, so other peers never see your prompts.

The four jobs and their stations

Logic node (green) lives inside a physical station scene (gray) placed in the world.

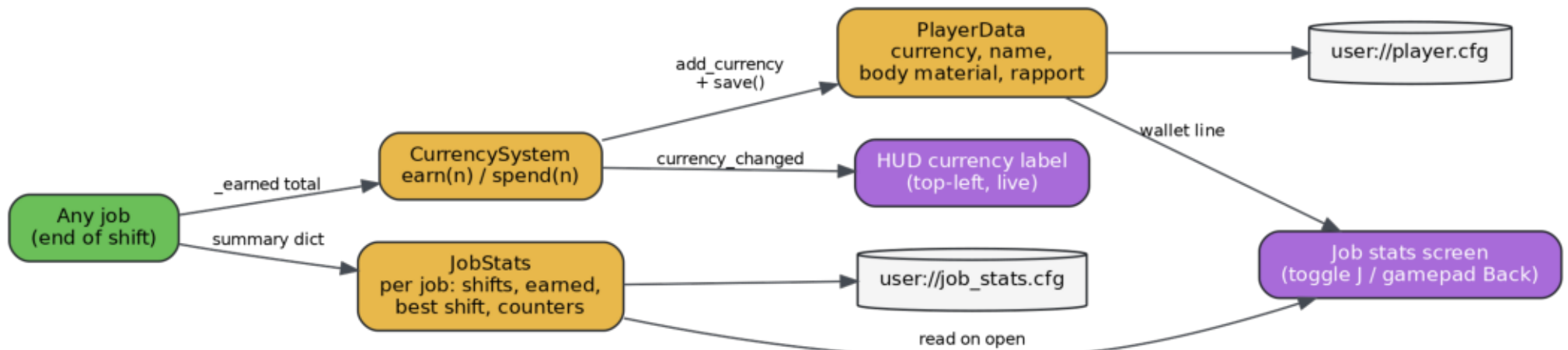


FOOD SERVICE — orders seat at tables; grab the right food crate item and bring it to THAT table before the ticket expires. RETAIL — two streams at once: shelves request restocks (carry the matching crate item over) while customers queue at the register (checkout with E).

DELIVERY — parcels queue at the depot, each addressed to a mailbox somewhere in the city; pay scales with distance plus a speed tip for beating par. CORPORATE — the only overlay job: a desk opens a spreadsheet UI where you approve or reject expense rows against a countdown. Wrong answers cost pay.

Economy, stats, persistence

Money and history both flow out of the same end-of-shift moment.

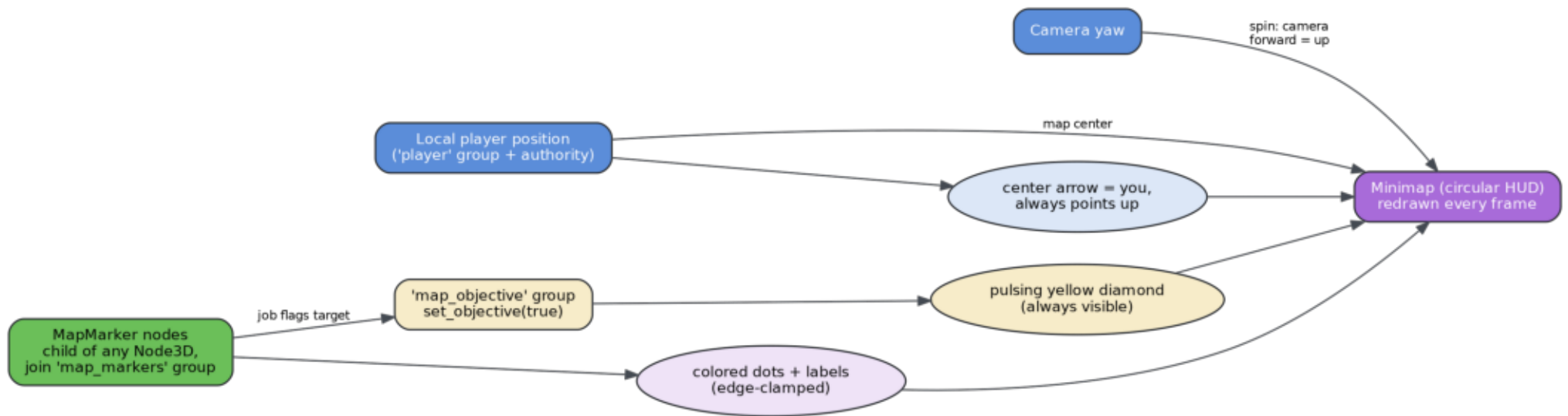


CURRENCY. Jobs never touch the wallet directly — they hand their shift total to `CurrencySystem.earn()`, which updates `PlayerData`, saves, and emits `currency_changed` so the HUD label updates the same frame. `spend()` refuses when the wallet is short.

STATS. `JobStats` sums every counter a job reports (served, missed, delivered, wrong...) and scores a performance rating from success vs failure counters. New jobs get stats automatically. Saves skip in headless runs so automated tests never pollute the real save files.

Minimap and objectives

Groups, not references: markers self-register, the minimap just reads groups each frame.



HOW MARKERS WORK. Drop a MapMarker node under anything and it appears on the map — no wiring, it joins the 'map_markers' group and the minimap scans that group when drawing. Each job station ships with one (food orange, retail green, depot blue, office purple, mailboxes cyan).

OBJECTIVES. A job flags where you should go with `set_objective(true)` — during a delivery the target mailbox pulses; between parcels the depot does. The diamond clamps to the map rim when out of range, so it always points the way. The map spins so camera- forward is up; the N tick rides the rim for global bearings.

Multiplayer: Steam lobby to replicated critters

Godot high-level multiplayer over SteamMultiplayerPeer. Authority = the peer who owns the critter.

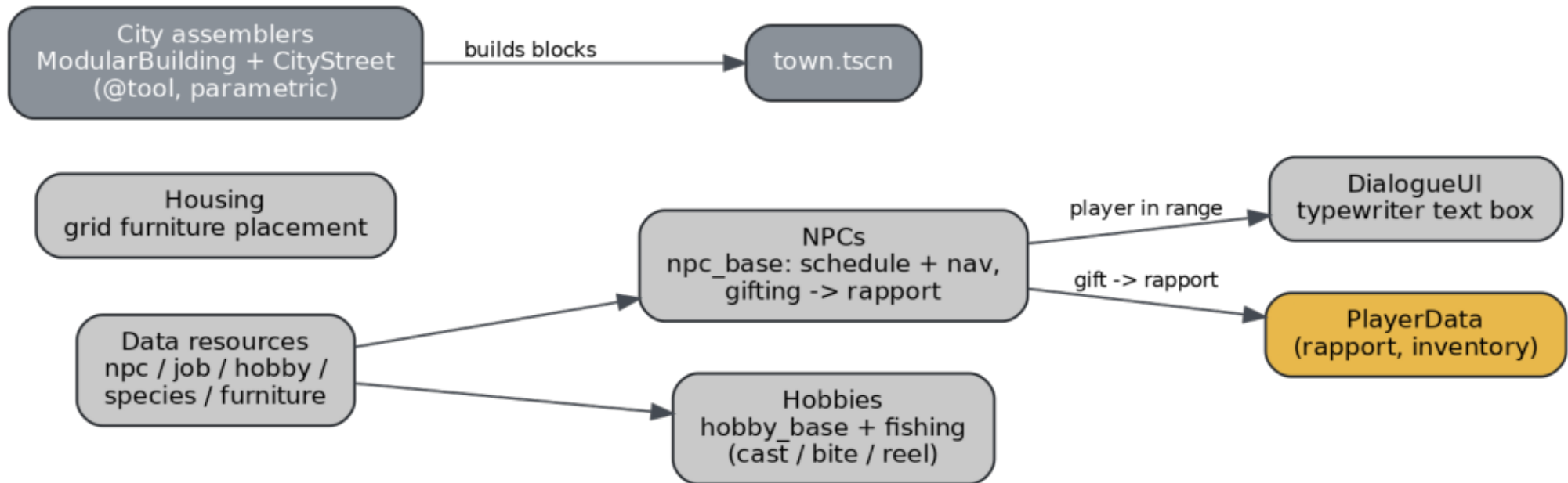


SESSION FLOW. SteamManager boots the Steam API at launch. The main menu drives the Multiplayer autoload to create or join a lobby; once connected it loads town.tscn on every peer, and the server's PlayerSpawnManager spawns one player node per peer id, replicated to all via MultiplayerSpawner.

AUTHORITY SPLIT. Your machine runs your critter's input, state machine, and physics. On everyone else's machine that critter is a proxy: physics off, position and animation state (NetAnimSync vars) streamed by MultiplayerSynchronizer. Carried item meshes are not replicated yet. No Steam? The spawner drops in a single local player so the game stays playable solo.

Worlds, city tools, and early prototypes

Gray = shipping world tech. Light gray = skeletons from the GDD awaiting content.



WORLDS. town.tscn is the main playable map — buildings assembled by parametric @tool scripts (ModularBuilding, CityStreet) from a glTF kit, so blocks are edited with exported numbers, not by hand. st_guy_blockout.tscn is a full village generated from a watabou map export using only CSG primitives — 46 labeled buildings, river, roads, trees — used as a second gameplay map.

PROTOTYPES. NPCs walk schedules via navigation and accept gifts that raise rapport (stored in PlayerData). Fishing implements the cast / wait / bite / react loop. Housing has a grid furniture-placement core. DialogueUI renders typewriter text. All compile and run but have no content pass yet — they are the next systems to grow.